Week 8

1. Three Pillars of Machine Learning

(a) **Linear Algebra :** A tool to understand the structure present in the data

(b) **Probability :** A tool to understand the uncertainity/noise present in the data

(c) **Optimization :** A tool to find the best model out off all possible models

2. Optimization

The following is the general form of an optimization problem

$$\begin{gathered}\min_x f(x)\\ s.t.\\ p_1(x)\leqslant 0,\ldots,p_n(x)\leqslant 0\\ q_1(x)=0,\ldots,q_m(x)=0\end{gathered}$$

- $f(x)$ is the objective function
- $p_1(x)\leqslant 0,\ldots,p_n(x)\leqslant 0$ are the inequality constraints

- $q_1(x) = 0, \ldots, q_m(x) = 0$ are the equality constraints
- $s.t.$ is the short form for **such that**

Any maximization problem can be framed as a minimization problem. Example :

$$\max_x \; -x^2 \equiv \min_x x^2$$

An optimization problem with atleast one constraint is called a **constrained optimization** problem and an optimization problem without any constraint is called an **unconstrained optimization** problem. In this week, we shall focus on unconstrained optimization.

Example

A company manufactures two products, **Product A** and **Product B**. Each unit of **Product A** requires 2 hours of machine time and 1 hour of labor, while each unit of **Product B** requires 1 hour of machine time and 3 hours of labor. The factory has at most 100 hours of machine time and 90 hours of labor available per day. Due to a contractual obligation, the company must produce exactly 30 units in total each day. The profit earned per unit is ₹40 for **Product A** and ₹50 for **Product B**. Additionally, due to market demand limitations, the company cannot produce more than 25 units of **Product B** in a day, and production quantities cannot be negative. The company wants to determine how many units of each product to produce in order to maximize total profit.

- X - Number of units of **Product A** produced in a day
- Y - Number of units of **Product B** produced in a day
- $40X + 50Y$ - Total profit per day
- $2X + Y$ - Number of machine time in hours per day
- $X + 3Y$ - Number of labor time in hours per day

Objective Function : $f(X, Y) = 40X + 50Y$

Constraints :

- $2X + Y \leqslant 100 \;\rightarrow\; 2X + Y - 100 \leqslant 0$
- $X + 3Y \leqslant 90 \;\rightarrow\; X + 3Y - 90 \leqslant 0$
- $Y \leqslant 25 \;\rightarrow\; Y - 25 \leqslant 0$
- $X + Y = 30 \;\rightarrow\; X + Y - 30 = 0$
- $X \geqslant 0 \;\&\; \rightarrow\; -X \leqslant 0$
- $Y \geqslant 0 \;\rightarrow\; -Y \leqslant 0$

Formulation as a Maximization Problem

$$\max_{X,Y} [40X + 50Y]$$
$$s.t.$$
$$2X + Y - 100 \leqslant 0$$
$$X + 3Y - 90 \leqslant 0$$
$$Y - 25 \leqslant 0$$
$$X + Y - 30 = 0$$
$$-X \leqslant 0$$
$$-Y \leqslant 0$$

Formulation as a Minimization Problem

$$\min_{X,Y} \; -[40X + 50Y]$$
$$s.t.$$
$$2X + Y - 100 \leqslant 0$$
$$X + 3Y - 90 \leqslant 0$$
$$Y - 25 \leqslant 0$$
$$X + Y - 30 = 0$$
$$-X \leqslant 0$$
$$-Y \leqslant 0$$

---

3. Taylor Series

One Variable Function $f : \mathbb{R} \to \mathbb{R}$

At a, if we happen to know $f(a)$, $f'(a)$, $f''(a)$, and so on, then $f(x)$ is given as follows :

$$f(x) = f(a) + f'(a)(x - a) + \frac{1}{2!} f''(a)(x - a)^2 + \ldots$$

If $x = a + \eta d$, where $\eta > 0$ and $d \in \mathbb{R}$, then

$$f(a + \eta d) = f(a) + f'(a)(\eta d) + \frac{1}{2!} f''(a)(\eta d)^2 + \ldots$$

Multi-Variable Function $f : \mathbb{R}^n \to \mathbb{R}$

At $v \in \mathbb{R}^n$, if we happen to know $f(v)$, $\nabla f(v)$, $H_f(v)$ and so on, then $f(x)$ is given as follows :

$$f(x) = f(v) + \nabla f(v)^T (x - v) + \frac{1}{2!} (x - v)^T H_f(v)(x - v) + \ldots$$

Here, $H_f(v)$ is the Hessian matrix of f at v

If $x = v + \eta d$, where $\eta > 0$ and $d \in \mathbb{R}^n$, then

$$f(v + \eta d) = f(v) + \nabla f(v)^T(\eta d) + \frac{\eta^2}{2!} d^T H_f(v) d + \ldots$$

---

4. Gradient Descent

Gradient descent is an iterative algorithm to solve an optimization problem.

Why Gradient Descent

Let us revisit linear regression. Every row of matrix X is a data point. Each data point is an element of $\mathbb{R}^d$. y is a $n \times 1$ column vector with the i^{th} row being the label of the i^{th} data point. Let us assume that the columns of X are linearly independent. The goal was to find a $\widehat{w}$ such that

$$\widehat{w} = \arg\min_w ||Xw - y||^2$$

The solution to the above optimization is as follows :

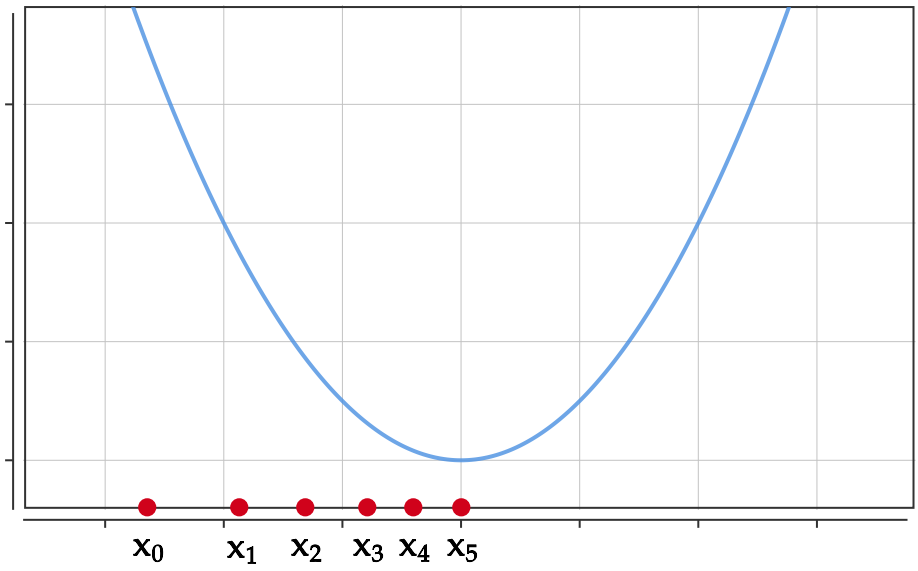
$$\widehat{w} = \left(X^T X\right)^{-1} X^T y$$

We have a nice, closed-form solution for $\widehat{w}$. By closed-form solution, we mean an exact answer to a problem that can be written using a finite combination of standard mathematical operations and functions, without requiring approximation or iterative methods.

Why do we need an iterative algorithm to solve an optimization problem?

- Sometimes, a nice, closed-form solution might not be available.
- Sometimes, even if a nice, closed-form solution is available, computing such a solution can be computationally costly. For example, $X^T X$ is a $d \times d$ matrix. If d is large, then finding the inverse for $X^T X$ can be computationally expensive.

The Algorithm (Informal)



We shall start at a point x_0 in the domain of the function. We take multiple steps to reach the optimal solution. Each step we take must get us closer to the optimal solution. From x_0, we move to x_1. We can say that x_1 is closer to the optimal solution than x_0 if $f(x_1) \leqslant f(x_0)$.

In the picture above, we started from x_0, moved to x_1, then to x_2 and all the way to x_5. With each step from x_0 to x_5, the function's value has reduced. From x_5, we cannot take any step that reduces the function's value. Therefore, we stop at x_5.

- In the above picture, each step is taken towards the right-hand side.
- However, not all steps are of equal magnitude. The distance between x_0 and x_1 is more than the distance between, x_4 and x_5
- We stop when no further step reduces the function's value.

The above three points highlight three important aspects of the iterative algorithm that we are trying to develop.

- In which direction do we have to move?
- How far should we move in the direction?
- When do we stop?

The following is an informal algorithm for finding optimal value of a function of one variable. Let $f : \mathbb{R} \rightarrow \mathbb{R}$.

- **Intialize** x_0
- **While** $\left(|f(x_{t+1}) - f(x_t)| > \epsilon\right)$
    - **Update** $x_{t+1} = x_t + \eta d$

- **Stop**

Let us observe the key aspects of the algorithm

- We start at x_0, which is arbitrarily chosen from $\mathbb{R}$.
- $t \in \{0, 1, 2, ... \}$
- x_t is the current position.
- We take a step from x_t to x_{t+1}
- d denotes the direction in which we have to take a step. d can be positive, or negative or zero.
- η, read as $'eta'$, decides how far do we have to move in the direction of d. $\eta > 0$
- As an example, if $x_t = 3$, $d = -2$ and $\eta = 0.1$, then $x_{t+1} = 3 - (0.1 \times 2) = 2.8$
- When a step is taken from x_t to x_{t+1}, if the absolute difference between $f(x_{t+1})$ and $f(x_t)$ is not greater than ϵ, we stop. Here, ϵ is a small positive number. Eg. 10^{-5}.
- Essentially, if we reach a point where the change in the function's value is extremely small, then it is better to stop. This usually happens when we are near the optimal value, where the derivative is close to 0. At such points, the function becomes almost flat. Additional steps only makes negligible difference to the function's value.

Key questions to ask:

- What should d be? It should be a direction along which the function's value reduces. If there exists no such direction, then, what should d be?
- What should η be? It decides how far do we have to move in the direction of d

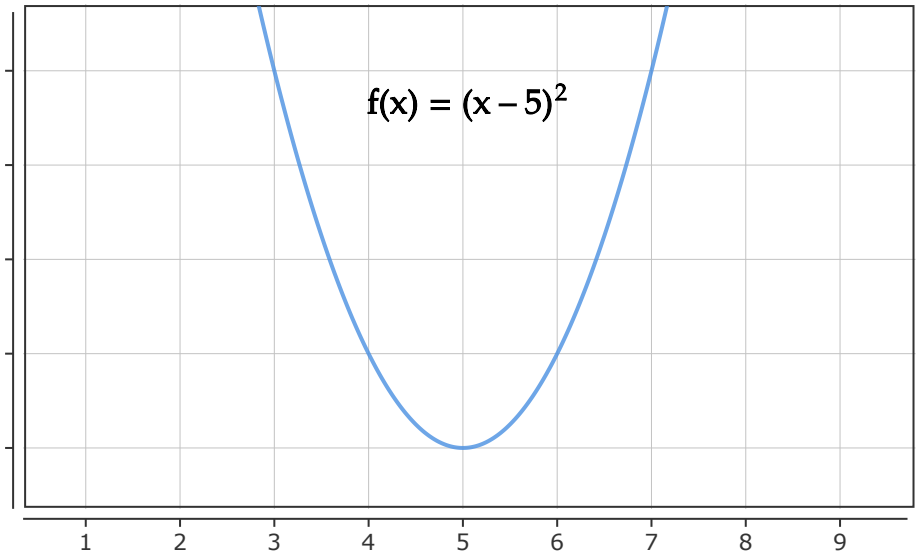
Before we answer these questions, let us also look at the informal algorithm for finding optimal value of a function of multiple variables. Let $f : \mathbb{R}^k \to \mathbb{R}$, $k > 1$.

- **Intialize** x_0
- **While** $\left(|f(x_{t+1}) - f(x_t)| > \epsilon\right)$
    - **Update** $x_{t+1} = x_t + \eta d$
- **Stop**

The algorithm is still the same. However,

- x_0 is arbitraily chosen from $\mathbb{R}^k$
- d is a vector in $\mathbb{R}^k$
- η is still a real number greater than 0

Good Direction



For $f(x) = (x-5)^2$, the minimum value of f occurs at $x = 5$.

Suppose, $x_t = 7$, then we would like to move towards the left-hand side. $x_{t+1} = 7 + \eta d$. Here, $\eta > 0$. To move towards the left hand side, d should be less than 0.

Suppose, $x_t = 3$, then we would like to move towards the right-hand side. $x_{t+1} = 3 + \eta d$. Once again, $\eta > 0$. To move towards the right hand side, d should be greater than 0

Suppose, $x_t = 5$, which is the minimum, then would like to stay at 5. $x_{t+1} = 5 + \eta d$. As $\eta > 0$, d should be equal to 0 so that $x_{t+1} = 5$. At this point, $x_{t+1} = x_t$, therefore, $|f(x_{t+1}) - f(x_t)| = 0$. The algorithm stops.

Our choice for d is $f'(x_t)$. The update rule is given as follows

$$x_{t+1} = x_t - \eta f'(x)$$

We will justify this choice later. For now, let us verify whether $d = -f'(x)$ satisfies the requirements listed above for $x_t = 7$, $x_t = 3$ and $x_t = 5$.

$f'(x)$ is given as follows.

$$f'(x) = 2x - 10$$

When $x_t = 7$

$$-f'(x_t) = -(14 - 10) = -4$$

When $x_t = 3$

$$-f'(x_t) = -(6 - 10) = 4$$

When $x_t = 5$

$$-f'(x_t) = -(10 - 10) = 0$$

$d = -f'(x_t)$ indeed satisfies the requirements listed above for $x_t = 7$, $x_t = 3$ and $x_t = 5$.

If $f : \mathbb{R}^k \to \mathbb{R}$, $k > 1$, then we our choice for d will be $\nabla f(x_t)$. The update rule is given as follows

$$x_{t+1} = x_t - \eta \nabla f(x_t)$$

Step Size

η is often referred to as the **step size**. First, let us see why do we need a step size. Through an example, we shall see what could potentially go wrong when we dont have any step size. Without η, the update rule is given as follows

$$x_{t+1} = x_t - f'(x_t)$$

Let $f(x) = (x - 5)^2$. Then, $f'(x) = 2x - 10$. Let us intialize $x_0 = 7$.

When $t = 0$

$$\begin{aligned} x_1 &= x_0 - f'(x_0) \\ &= 7 - f'(7) \\ &= 7 - 4 \\ &= 3 \end{aligned}$$

When $t = 1$

$$\begin{aligned} x_2 &= x_1 - f'(x_1) \\ &= 3 - f'(3) \\ &= 3 + 4 \\ &= 7 \end{aligned}$$

We are stuck in a loop. $x_0 = 7$, $x_1 = 3$, $x_2 = 7$. As a result, $x_3 = 3$, $x_4 = 7$, $x_5 = 3$ and so on.

Recall, the $f(x)$ attains its minimum at $x = 5$. At $x = 7$, the direction is -4. Although the direction is correct, the step is too large, causing us to move from 7, bypassing 5 and reaching 3.

Similarly, at $x = 3$, the direction is 4. Again, the direction is correct. But, we bypass 5 and reach 7.

By setting a suitable value for η, we can ensure that we take small steps in the right direction, and eventually reach the optimal point. What is a good choice for η?

Let us first look at a bad choice for η.

Bad Choice : $\eta_t = \frac{1}{2^t}$, $t \in \{0, 1, 2, \ldots\}$. Update rule :

$$x_{t+1} = x_t - \eta_t f'(x_t)$$

$$= x_t - \frac{f'(x_t)}{2^t}$$

As t increases, $\frac{1}{2^t}$ becomes extremely close to 0. Consequently, the update from x_t to x_{t+1} becomes negligible, resulting in almost no change between one step and the next. This makes progress towards the minimum extremely slow. The absolute difference between $f(x_{t+1})$ and $f(x_t)$ will become extremely close to 0, thereby, forcing the algorithm to stop even before convergence.

Good Choice : $\eta = \frac{1}{t+1}$, $t \in \{0, 1, 2, \ldots\}$. Update rule :

$$x_{t+1} = x_t - \eta_t f'(x_t)$$

$$= x_t - \frac{f'(x_t)}{t+1}$$

Unlike $\frac{1}{2^t}$, this does not decay exponentially. The step size starts at 1 and decreases slowly, thereby allowing meaningful progress for many iterations. Since it decreases, it ensures that the algorithm does not take large steps and bypass the optimal value. The gradual shrinkage of the step size ensures that the algorithm eventually converges to a point rather than oscillating.

Why $d = -f'(x_t)$

For $f : \mathbb{R} \rightarrow \mathbb{R}$, the update rule is as follows

$$x_{t+1} = x_t + \eta_t d$$

Let us express $f(x_{t+1})$ using Taylor series.

$$f(x_t + \eta d) = f(x_t) + f'(x_t)(\eta d) + \underbrace{\frac{1}{2!} f''(x_t)(\eta d)^2 + \ldots}_{\leftarrow \textit{Higher order terms} \rightarrow}$$

The higher order terms contribute minimally to the overall sum. We can therefore, approximate $f(x_t + \eta d)$ as follows

$$\begin{aligned} f(x_t + \eta d) &\approx f(x_t) + f'(x_t)(\eta d) \\ f(x_t + \eta d) - f(x_t) &\approx f'(x_t)(\eta d) \\ f(x_{t+1}) - f(x_t) &\approx f'(x_t)(\eta d) \end{aligned}$$

As mentioned before, when we take a step from x_t to x_{t+1}, we would like to have $f(x_{t+1}) \leqslant f(x_t)$

$$\begin{aligned} f(x_{t+1}) &\leqslant f(x_t) \\ f(x_{t+1}) - f(x_t) &\leqslant 0 \\ f'(x_t)(\eta d) &\leqslant 0 \\ f'(x_t)(d) &\leqslant 0 \end{aligned}$$

$\eta > 0$, therefore, $f'(x_t)(d)$ should be less than 0. Notice that d depends on $f'(x_t)$. If $f'(x)$ is positive, d should be negative and if $f'(x)$ is negative, then d should be positive. If $d = -f'(x_t)$, then

$$f'(x_t)(d) = -[f'(x_t)]^2$$

$[f'(x_t)]^2 \geqslant 0$, therefore, $-[f'(x_t)]^2 \leqslant 0$.

Why $d = -\nabla f(x_t)$

For $f : \mathbb{R}^k \rightarrow \mathbb{R}$, $k > 1$, the update rule is as follows

$$x_{t+1} = x_t + \eta_t d$$

Let us express $f(x_{t+1})$ using Taylor series.

$$f(x_t + \eta d) = f(x_t) + \nabla f(x_t)^T(\eta d) + \underbrace{\frac{\eta^2}{2!} d^T H_f(v) d + \ldots}_{\leftarrow \textit{Higher order terms} \rightarrow}$$

The higher order terms contribute minimally to the overall sum. We can therefore, approximate $f(x_t + \eta d)$ as follows

$$\begin{aligned} f(x_t + \eta d) &\approx f(x_t) + \nabla f(x_t)^T(\eta d) \\ f(x_t + \eta d) - f(x_t) &\approx \nabla f(x_t)^T(\eta d) \\ f(x_{t+1}) - f(x_t) &\approx \nabla f(x_t)^T(\eta d) \end{aligned}$$

As mentioned before, when we take a step from x_t to x_{t+1}, we would like to have $f(x_{t+1}) \leqslant f(x_t)$

$$\begin{aligned} f(x_{t+1}) &\leqslant f(x_t) \\ f(x_{t+1}) - f(x_t) &\leqslant 0 \\ \nabla f(x_t)^T(\eta d) &\leqslant 0 \\ \nabla f(x_t)^T(d) &\leqslant 0 \end{aligned}$$

$\eta > 0$, therefore, $\nabla f(x_t)^T(d)$ should be less than 0. There are infinitely many $d \in \mathbb{R}^k$ satisfying $\nabla f(xt)^T d \leqslant 0$. Recall that $-\nabla f(x_t)$ is the direction of the **steepest descent** of $f(x)$ at x_t. Therefore, we shall choose $d = -\nabla f(x_t)$

$$\nabla f(x_t)^T(-\nabla f(x_t)) = -||\nabla f(x_t)||^2$$

$||\nabla f(x_t)||^2 \geqslant 0$, therefore, $-||\nabla f(x_t)||^2 \leqslant 0$.

Gradient Descent Algorithm

Let $\eta_t = \dfrac{1}{t+1}$, $t \in \{0, 1, 2, \ldots.\}$

Let $f : \mathbb{R} \rightarrow \mathbb{R}$

- **Intialize** x_0

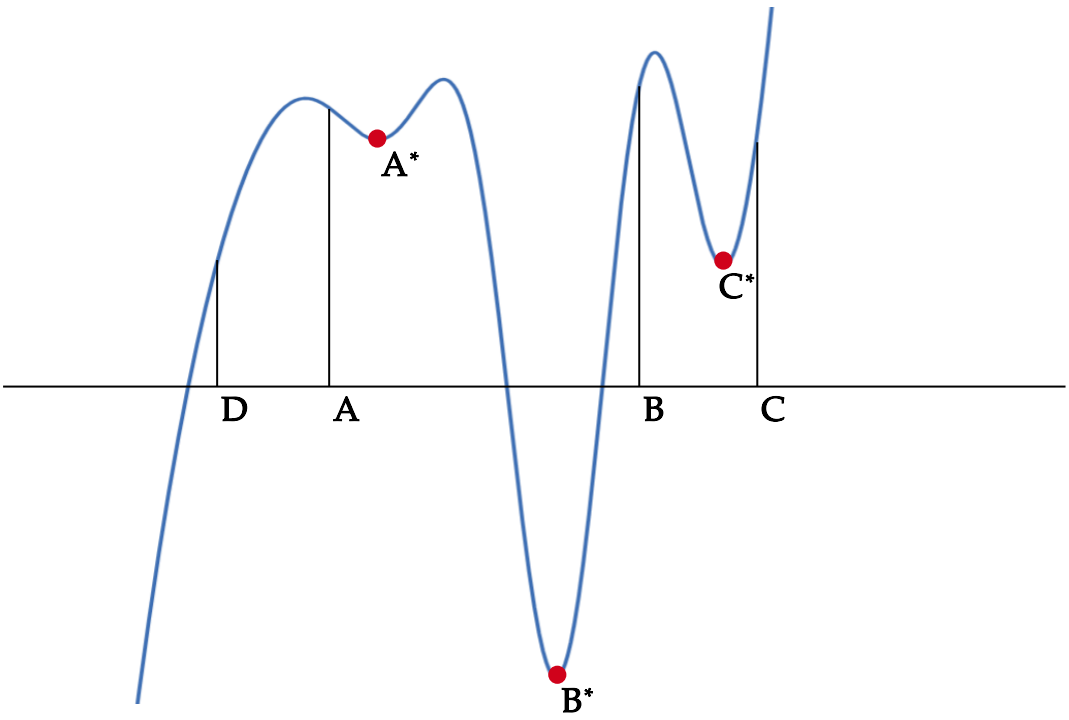
- **While** $\left(|f(x_{t+1}) - f(x_t)| > \epsilon\right)$
  - **Update** $x_{t+1} = x_t - \eta_t f'(x_t)$
- **Stop**

Let $f : \mathbb{R}^k \to \mathbb{R}, k > 1$

- **Intialize** x_0
- **While** $\left(|f(x_{t+1}) - f(x_t)| > \epsilon\right)$
  - **Update** $x_{t+1} = x_t - \eta_t \nabla f(x_t)$
- **Stop**

Convergence of Gradient Descent Algorithm

Gradient descent algorithm converges to a **local minimum**, subject to the initial value x_0. It need not converge to a global minimum. The choice of the initial value x_0 significantly influences the convergence of the algorithm.



- If $x_0 = \mathbf{A}$, the algorithm converges to $\mathbf{A}^*$
- If $x_0 = \mathbf{B}$, the algorithm converges to $\mathbf{B}^*$
- If $x_0 = \mathbf{C}$, the algorithm converges to $\mathbf{C}^*$
- If $x_0 = \mathbf{D}$, the algorithm diverges to $-\infty$